

# Game Client Programmer DirectX Portfolio

이종혁

jhorn3927@gmail.com

<https://youtu.be/MyqkYV1b1fc>

[github.com/Impersy](https://github.com/Impersy)

# About Me

## Profile

전투 시스템 중심의 Gameplay Programming에 관심이 있는 게임 클라이언트 프로그래머입니다.

DirectX 기반 개인/팀 프로젝트를 통해 전투, 카메라, UI, 렌더링 시스템을 구현했습니다.

## Focus Area

Combat Systems · Gameplay Programming · Rendering Systems

## Strengths

문제를 시스템 단위로 분해하고, 게임플레이에 필요한 기능을 구조적으로 구현합니다.

수학/좌표계 기반 구현(Camera/Minimap)과 최적화 포인트(Instancing)를 경험했습니다.

## Skills

### [Language]

C++

### [Graphics]

DirectX9 / DirectX11

### [Systems]

Deferred Rendering

OBB Collision

Particle Instancing

Combat Systems

Camera Systems

# Projects Overview

## [Demon Slayer Project]

DirectX11 **Team** Project (6인/60일)

Camera System / Awakening System / UI Systems / VFX(젠이츠, CutScene)

<https://github.com/Impersy/DirectX11-DemonSlayer-Project.git>

## [Thymesia Project]

DirectX11 **Personal** Project (1인/43일)

Parry System / Particle Instancing / OBB Collision

<https://github.com/Impersy/DirectX11-Thymesia-Project.git>

## [IRA Project]

DirectX9 **Team** Project (4인/30일)

Skill System / Player Combat / Camera System

<https://github.com/Impersy/DirectX9-IRA-Project.git>

# [Demon Slayer Project]

## [Project]

Demon Slayer Recreation

## [Project Info]

Engine : DirectX11

Development Period : 60 Days

Team Size : 6

## [My Core Implementation]

Combat Camera System

Awakening Cutscene System

Minimap UI System



# [Demon Slayer Project] - Combat Camera System

## Feature

전투 상황에 따라 카메라 시점을 동적으로 전환하는 전투 카메라 시스템

## How it Works

플레이어-적 전투축을 기반으로 카메라 방향 계산

플레이어-적 중심(BattleCenter)을 바라보도록 시선 처리

Combo & 근거리 전투 상황에서만 Side View로 전환

Lerp 기반 보간으로 카메라 움직임 안정화

## Result

Player-Enemy 중심 프레이밍을 사용해 전투 대상 가시성 확보

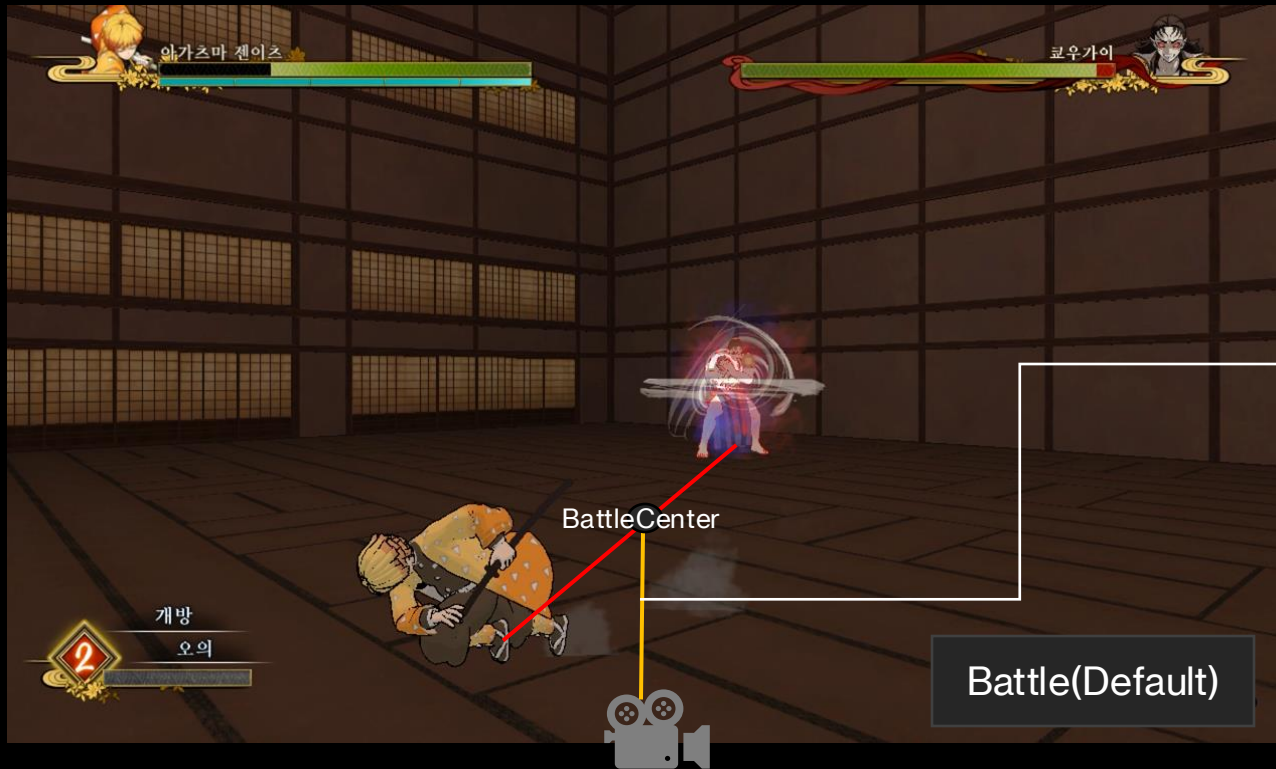
Combo 상황에서 **Side Camera**로 전환하여 공격 흐름 강조

카메라 이동과 시선 방향을 Lerp로 보간해 화면 안정성 유지

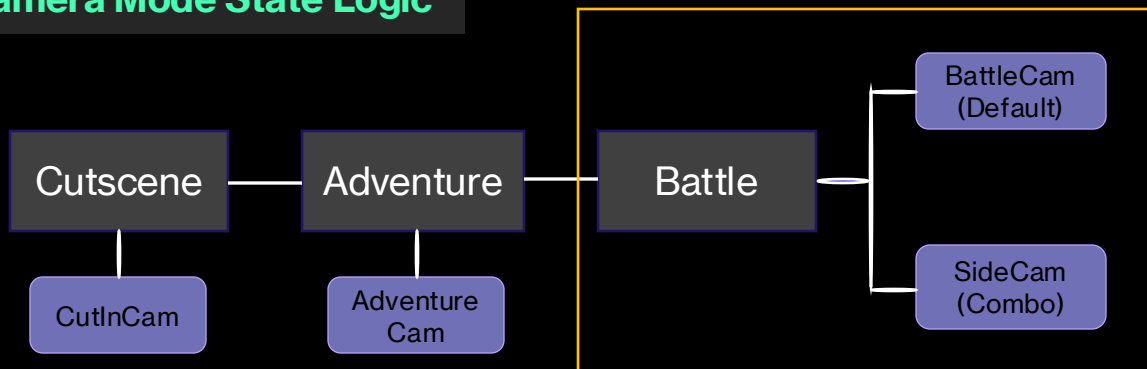




## Battle Camera Logic



## Camera Mode State Logic



## Battle Camera Update

```

// 1) Player-Enemy 상대 벡터를 기준으로 전투 카메라 방향 계산
_vector vUp = XMVector3Normalize({ 0.f, 1.f, 0.f });
_vector vCamPosition =
    m_pTransformCom->Get_State(CTransform::STATE_POSITION);

if (m_bIs_Dist_Update)
{
    // Player → Enemy 방향 벡터
    m_vDist = m_vTargetPos - m_vBattleTargetPos;

    // Y축으로 일정 각도 회전한 벡터를 카메라 배치 방향으로 사용
    _matrix RotationMatrix =
        XMMatrixRotationAxis(vUp, XMConvertToRadians(-m_vCameraAngle));
    m_vDist = XMVector3TransformNormal(m_vDist, RotationMatrix);

    // 수직 높이 고정 및 정규화
    m_vDist = { XMVectorGetX(m_vDist), 0.1f,
        XMVectorGetZ(m_vDist), XMVectorGetW(m_vDist) };
    m_vDist = XMVector3Normalize(m_vDist);
}

// 2) 계산된 방향 벡터와 거리 값을 이용해 목표 카메라 위치 계산
_vector vDest = m_vTargetPos + m_vOffSet + (m_vDist * m_fDistance);
_float t = (_float)dTimeDelta * m_fDamping;
_vector CamPos = XMVectorLerp(vCamPosition, vDest, t);

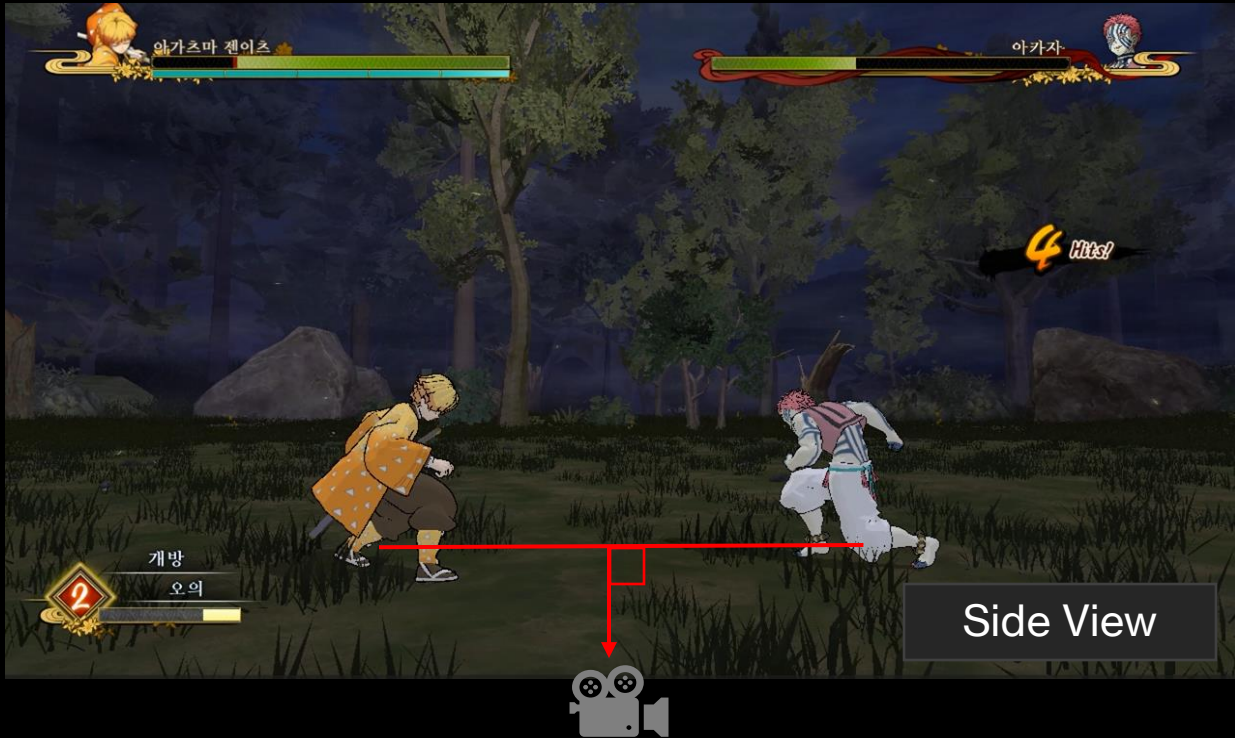
m_pTransformCom->Set_State(CTransform::STATE_POSITION, CamPos);

// 3) 시선은 Player와 Enemy의 중심 지점(BattleCenter)을 향하도록 설정
_vector NewLook = m_vBattleCenter + m_vLookOffSet - CamPos;

NewLook = { XMVectorGetX(NewLook), XMVectorGetY(NewLook) * 0.f,
    XMVectorGetZ(NewLook), XMVectorGetW(NewLook) };
NewLook = XMVector3Normalize(NewLook);

// 4) Look 방향도 부드럽게 보간하여 카메라 움직임을 안정화
_float New_t = (_float)dTimeDelta * m_fLookDamping;
m_pTransformCom->LerpVector(NewLook, New_t);
  
```

## Side Camera Logic



## Troubleshooting

### Problem

벽력일섬 사용 시 플레이어가 적을 관통하면서 Player-Enemy 전투축이 반전되고, 그 결과 Side Camera가 반대 방향으로 급회전하는 문제가 발생

### Fix

외적 계산에 사용되는 전투축의 부호를 `m_Hekireki_Dir`로 보정해, 스킬 사용 후에도 Side Camera가 의도한 방향을 유지하도록 개선

### Side Camera Update

```
// 1) Player-Enemy 전투축을 기준으로 측면 카메라 방향 계산
_vector vUp = XMVector3Normalize({ 0.f, 1.f, 0.f });
_vector vCamPosition =
    m_pTransformCom->Get_State(CTransform::STATE_POSITION);

if (m_bIs_Dist_Update) {
    _vector vRight = m_vTargetPos - m_vBattleTargetPos;
    // 전투축과 Up 벡터의 외적을 사용해 Side View 방향을 생성
    m_vDist = XMVector3Cross(m_Hekireki_Dir * vRight, vUp);
    m_vDist = { XMVectorGetX(m_vDist), 0.f,
        XMVectorGetZ(m_vDist), XMVectorGetW(m_vDist) };
    m_vDist = XMVector3Normalize(m_vDist);

    _matrix RotationMatrix =
        XMMatrixRotationAxis(vUp, XMConvertToRadians(8.f));
    m_vDist = XMVector3TransformNormal(m_vDist, RotationMatrix);
}

// 2) 계산된 측면 방향을 기준으로 BattleCenter 주변에 카메라 배치
_vector vDest = m_vBattleCenter + m_vOffSet + (m_vDist * m_fDistance);
_float t = (_float)dTimeDelta * m_fDamping;
_vector CamPos = XMVectorLerp(vCamPosition, vDest, t);
m_pTransformCom->Set_State(CTransform::STATE_POSITION, CamPos);

// 3) 시선은 BattleCenter를 향하도록 유지해 콤보 동작이 화면 중심에 오도록 처리
_vector NewLook = m_vBattleCenter + m_vLookOffSet - CamPos;

NewLook = { XMVectorGetX(NewLook), 0.5f,
    XMVectorGetZ(NewLook), XMVectorGetW(NewLook) };
NewLook = XMVector3Normalize(NewLook);

_float New_t = (_float)dTimeDelta * m_fLookDamping;
m_pTransformCom->LerpVector(NewLook, New_t);
```

# [Demon Slayer Project] - Awakening Cutscene System

## Feature

각성 연출을 다중 카메라 컷 시퀀스로 구성  
카메라 연산과 컷 시퀀스 제어를 분리한 구조

## How it Works

컷 데이터(CutInCamDesc)에 각도 / 거리 / 오프셋 / 지속시간 등을 정의  
CutInFinish() → 단일 컷 카메라 연산  
CutInCamera() → 컷 시퀀스 재생 및 전환

## Result

데이터 기반 컷 구성으로 캐릭터별 연출을 코드 수정 없이 확장 가능  
컷신 종료 후 전투 카메라로 자연스럽게 복귀

```
public:
    enum CUT_IN_FINISH
    { TANJIRO_AWAKE, ZENITSU_AWAKE, AKAZA_AWAKE,
      KYOGAI_AWAKE, TANJIRO_EXECUTION, END_FINISH };
```

→ 캐릭터별 enum



```
typedef struct Cut_In_Camera_Desc
{
    _bool bTarget_Is_Player = {};
    _bool bLookTarget_Is_Player = {};
    _float Angle_Hori = {};
    _float Angle_Verti = {};
    _float fDistance = {};
    _float fLifeTime = {};
    _bool bIs_Lerp = {};
    _vector vOffSet = {};
    _vector vLookOffSet = {};
}CutInCamDesc;
```

```
private:
    vector<CutInCamDesc> m_Cut_In_Finish[END_FINISH];
```

→ 컷 데이터 구조체

→ 캐릭터별 컷 시퀀스를 저장하는 vector 배열



## CutInFinish( ) -> 단일 컷 카메라 연산

### Role

컷 데이터(CutInCamDesc)를 기반으로 단일 컷의 카메라 연산을 수행

### How it Works

대상(Player / Enemy) 기준으로 카메라 방향 계산

수평 / 수직 회전으로 컷 앵글 생성

거리 / 오프셋 반영 후 카메라 위치 결정

필요 시 Lerp로 부드럽게 이동

LookAt으로 시선 설정

### Design Point

컷 연출 차이를 로직 분기 대신 데이터 조합으로 제어하도록 설계

```
void CCamera_Free::CutInFinish(_double dTimeDelta, const CutInCamDesc& Desc)
{
    // 1. 대상(Player / Enemy) 기준 기본 방향 계산
    if (Desc.bTarget_Is_Player) m_vDist = m_vCutInTargetPos - m_vTargetPos;
    else m_vDist = m_vCutInBattleTargetPos - m_vBattleTargetPos;
    m_vDist = XMVector3Normalize(m_vDist);

    // 2. 수직 / 수평 회전으로 컷 앵글 생성
    _vector vUp = XMVector3Normalize({ 0.f,1.f,0.f });
    _vector vRight = XMVector3Normalize(XMVector3Cross(vUp, m_vDist));

    _matrix RotationMatrix = XMMatrixRotationAxis(vRight, XMConvertToRadians(Desc.Angle_Verti));
    m_vDist = XMVector3TransformNormal(m_vDist, RotationMatrix);

    RotationMatrix = XMMatrixRotationAxis(vUp, XMConvertToRadians(Desc.Angle_Hori));
    m_vDist = XMVector3TransformNormal(m_vDist, RotationMatrix);

    // 3. 거리 / 오프셋을 반영해 목표 카메라 위치 계산
    _vector vDest = {};
    if (Desc.bTarget_Is_Player)
        vDest = m_vTargetPos + m_vOffset + Desc.vOffset + (m_vDist * Desc.fDistance);
    else
        vDest = m_vBattleTargetPos + m_vOffset + Desc.vOffset + (m_vDist * Desc.fDistance);

    // 4. 옵션에 따라 즉시 이동 또는 보간 이동
    if (Desc.bIs_Lerp == true) {
        _float t = (_float)dTimeDelta * m_fDamping;
        _vector CamPos = XMVectorLerp(m_pTransformCom->Get_State(CTransform::STATE_POSITION), vDest, t);
        m_pTransformCom->Set_State(CTransform::STATE_POSITION, CamPos);
    }
    else {
        m_pTransformCom->Set_State(CTransform::STATE_POSITION, vDest);
    }

    // 5. 컷 데이터 기준으로 시선 처리
    if (Desc.bLookTarget_Is_Player)
        m_pTransformCom->LookAt(m_vTargetPos + m_vLookOffset + Desc.vLookOffset + Desc.vOffset);
    else
        m_pTransformCom->LookAt(m_vBattleTargetPos + m_vLookOffset + Desc.vLookOffset + Desc.vOffset);
}
```

→ 데이터 기반으로 카메라 제어

# CutInCamera( ) → Cutscene Sequencing

## Role

컷 시퀀스 재생 및 전환 제어를 담당

## How it Works

현재 컷 인덱스에 해당하는 컷 데이터 선택

CutInFinish() 호출로 단일 컷 연산 수행

LifeTime 기준으로 다음 컷으로 전환

마지막 컷 종료 시 전투 카메라 상태 복귀

## Design Point

카메라 연산(CutInFinish)과 컷 시퀀스 제어(CutInCamera)를

분리해 역할을 명확히 구성

```
void CCamera_Free::CutInCamera(_double dTimeDelta)
{
    // 1. 현재 컷 인덱스에 해당하는 카메라 컷 데이터 선택
    CutInCamDesc Cam =
        m_Cut_In_Finish[m_Cut_In_Finish_Type][m_Cut_In_Finish_Cam_Num];

    // 2. 실제 단일 컷 카메라 연산은 CutInFinish()에 위임
    CutInFinish(dTimeDelta, Cam);

    // 3. 현재 컷의 재생 시간 누적
    m_Cut_In_Finish_TimeAcc += (_float)dTimeDelta;

    // 4. 컷의 LifeTime이 끝나면 다음 컷으로 전환
    if (m_Cut_In_Finish_TimeAcc > Cam.fLifeTime) {

        m_Cut_In_Finish_TimeAcc = 0.f;
        m_Cut_In_Finish_Cam_Num++;

        // 5. 마지막 컷까지 모두 재생된 경우 컷신 종료
        if (m_Cut_In_Finish_Cam_Num > m_Cut_In_Finish[m_Cut_In_Finish_Type].size() - 1) {

            전투 카메라 기본 파라미터 재설정

            // 전투 카메라 위치 복귀
            _vector vDest = m_vTargetPos + m_vOffset + (m_vDist * m_fDistance);
            m_pTransformCom->Set_State(CTransform::STATE_POSITION, vDest);
            // 전투 중심을 바라보도록 시선 복귀
            _vector NewLook = m_vBattleCenter + m_vLookOffset;
            m_pTransformCom->LookAt(NewLook);

            // 컷신 상태 종료 처리
            m_Cut_In_Finish_Cam_Num = 0;
            m_Cut_In_IsDone = true;
            m_Is_Cut_In = false;
            return;
        }
    }
}
```

# [Demon Slayer Project] - Minimap System

## Feature

플레이어의 현재 위치와 진행 방향을 확인할 수 있는 동적 미니맵 UI

## How it Works

플레이어 월드 좌표를 맵별 UV 좌표로 정규화

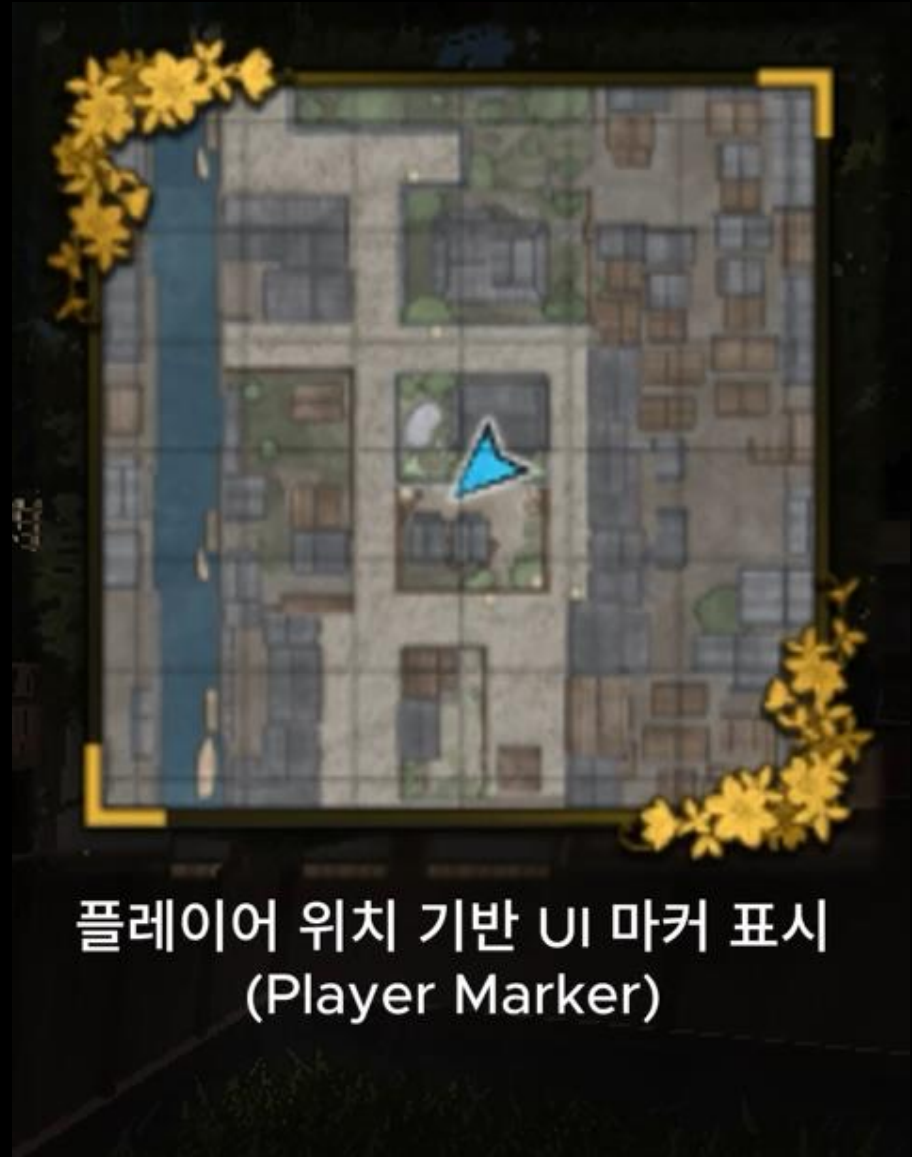
UV 중심 좌표를 기준으로 미니맵 표시 영역 갱신

플레이어 Look 벡터를 기반으로 아이콘 회전 적용

## Result

전체 맵이 아닌 주변 영역만 표시해 미니맵 가독성 향상

플레이어 위치와 방향을 즉시 파악할 수 있어 탐색 편의성 강화



## World Position → UV Mapping

### Concept

플레이어의 월드 좌표를 맵 기준 UV 좌표로 변환

### How it Works

플레이어 월드 좌표 (x, z) 추출

맵별 월드 범위를 기준으로 **0~1 UV** 좌표로 정규화

계산된 UV 값을 미니맵 중심 좌표로 사용

World Space

Player Pos

Normalize

UV Space

UV Center

```
if (m_Map_Type == 1) {  
  
    // player Pos  
    _vector Pos = m_pTargetTransformCom->Get_State(CTransform::STATE_POSITION);  
  
    // World Position → UV coordinate  
    m_UV_Centor_X = XMVectorGetX(Pos);  
    m_UV_Centor_X = (m_UV_Centor_X - 35) / (95 - 29);  
  
    m_UV_Centor_Y = XMVectorGetZ(Pos);  
    m_UV_Centor_Y = (m_UV_Centor_Y - 1) / (79 - 1);  
  
    // Player icon position  
    m_UV_Player_X = m_UV_Centor_X;  
    m_UV_Player_Y = m_UV_Centor_Y;  
}
```

월드 좌표 범위를 기준으로 플레이어 위치를 0~1 UV 좌표로 정규화해 미니맵 중심 좌표를 계산  
맵마다 월드 크기가 다르기 때문에, 각 레벨에 맞는 좌표 범위를 기준으로 UV 변환을 수행



# UV Crop Rendering

## Concept

플레이어 중심 UV 좌표를 기준으로 미니맵 표시 영역을 계산

## How it Works

UV\_Cut\_Min/Max 로 표시 범위 계산

중심 좌표가 맵 가장자리에 가까워지면 clamp 처리

셰이더에서 범위 밖 픽셀을 **alpha cut** 처리



해당 프레임 안쪽만  
Rendering

```
// 1) Player 중심 UV 좌표를 기준으로 미니맵에 표시할 영역(window) 계산
m_UV_Cut_MinX = m_UV_Centor_X - 0.3f;
m_UV_Cut_MaxX = m_UV_Centor_X + 0.3f;
```

```
m_UV_Cut_MinY = (1 - m_UV_Centor_Y) - 0.3f;
m_UV_Cut_MaxY = (1 - m_UV_Centor_Y) + 0.3f;
```

```
// 2) UV 범위가 0~1 밖으로 벗어나지 않도록 clamp 처리
```

```
if (m_UV_Cut_MinX <= 0.f) m_UV_Cut_MinX = 0.f;
if (m_UV_Cut_MaxX >= 1.f) m_UV_Cut_MaxX = 1.f;
```

```
if (m_UV_Cut_MinY <= 0.f) m_UV_Cut_MinY = 0.f;
if (m_UV_Cut_MaxY >= 1.f) m_UV_Cut_MaxY = 1.f;
```

```
PS_OUT PS_MAIN_ALPHA_CUT(PS_IN In)
```

```
{
```

```
PS_OUT Out = (PS_OUT)0;
```

```
vector vColor = g_Texture.Sample(LinearSampler, In.vTexUV);
```

```
// UV cut 범위를 벗어난 영역은 alpha를 제거
```

```
if (In.vTexUV.x <= g_UV_Cut_MinX) vColor.w *= 0.f;
```

```
if (In.vTexUV.x >= g_UV_Cut_MaxX) vColor.w *= 0.f;
```

```
if (In.vTexUV.y <= g_UV_Cut_MinY) vColor.w *= 0.f;
```

```
if (In.vTexUV.y >= g_UV_Cut_MaxY) vColor.w *= 0.f;
```

```
vColor.w *= g_Alpha;
```

```
Out.vColor = vColor;
```

```
return Out;
```

```
}
```

HLSL

CPU에서 UV 좌표를 계산하고, GPU 셰이더에서 범위 밖 영역을 제거하는 방식으로 동적인 미니맵 표시 영역을 구현했다.

# [Thymesia Project]

## [Project]

Thymesia Recreation

## [Project Info]

Engine : DirectX11

Personal Project

Development Period : 43 Days

## [My Core Implementation]

Parry System (KeyFrame)

Particle Instancing

Weapon Hit Detection (OBB Collision)



# [Thymesia Project] - Parry System

## Feature

보스 공격의 애니메이션 / KeyFrame / 충돌 상태를 함께 검사해 타이밍 기반 패링이 가능하도록 구현

## How it Works

보스의 현재 공격 애니메이션과 KeyFrame을 확인

충돌이 발생한 상태에서 패링 가능 프레임 구간이면 성공 처리

성공 시 Deflect 애니메이션과 패링 이펙트를 재생

## Result

공격마다 다른 패링 타이밍을 구성할 수 있었고,  
성공 시 즉각적인 반응 애니메이션과 이펙트로 전투 피드백을 강화했다.





# Parry Timing Detection

## Role

보스 공격별 패링 가능 타이밍 구간 판정 시스템

## How it Works

현재 보스 공격 애니메이션(m\_Boss\_Anim\_index) 확인

현재 공격 프레임(m\_Boss\_KeyFrame)이 패링 가능 구간인지 검사

충돌(m\_Is\_Coll)과 패링 입력이 동시에 만족되면 성공 처리

## Design Point

공격마다 다른 패링 타이밍을 **KeyFrame** 으로 제어하도록 구성

```
void CPlayer::Parry_From_Boss()
{
    CGameInstance* pGameInstance = CGameInstance::GetInstance();
    Safe_AddRef(pGameInstance);

    // 1. 플레이어가 피격 애니메이션 상태가 아닐 때만 패링 판정 수행
    if ((m_Anim_index < 11 || m_Anim_index > 16))
    {
        // 2. 보스 무기와 실제 충돌 중인지 확인
        if (m_Is_Coll) {

            // 3. 현재 보스 공격 애니메이션 종류 확인
            if (m_Boss_Anim_index == BOSS_ATK_1) {

                // 4. 해당 공격의 패링 가능 KeyFrame 구간인지 검사
                if (m_Boss_KeyFrame >= 52 && m_Boss_KeyFrame <= 60) {

                    // 5. 패링 입력이 유지되고 있으면 패링 성공 처리
                    if (pGameInstance->Key_Pressing(KEY_RBUTTON)) {
                        m_Parry_Deflect_Lock = true; // 중복 패링 방지 플래그
                        m_pModelCom->Reset_TimeAcc_Channel(); // 현재 애니메이션 시간 누적값 초기화

                        // Deflect 애니메이션 4종 중 하나 랜덤 선택
                        _uint index = rand() % 4;
                        m_Anim_index = 32 + index;

                        // Deflect 루트모션을 적용할 기준 좌표계 저장
                        m_World = m_pTransformCom->Get_WorldFloat4x4();

                        // 패링 반응 애니메이션 도중 플레이어 입력 잠금
                        m_Key_Lock = true;

                        Safe_Release(pGameInstance);
                        return;
                    }
                }
            }
        }
    }
}
```



## Directional Parry Effect

### Role

패링 성공 시 **방향성 스파크와 라이트** 피드백 생성

### How it Works

Deflect 애니메이션 방향에 따라 파티클 타입 지정

패링 위치와 월드행렬을 전달해 방향성 이펙트 생성

Tick에서 빛과 크기를 갱신해 순간적인 타격감 연출

실제 파티클 확산은 공용 인스턴싱 버퍼 로직에서 처리

### Design Point

패링 이펙트 오브젝트는 **피드백 제어**에 집중하고

실제 파티클 확산 계산은 **공용 인스턴싱 버퍼 로직**에 위임

```
// Deflect 애니메이션 방향에 맞는 Parry 타입 지정  
CParry_Particle::PARRYDESC ParryDesc;
```

```
if (m_Anim_index == 32) ParryDesc.Type = 1;  
if (m_Anim_index == 33) ParryDesc.Type = 2;  
if (m_Anim_index == 34) ParryDesc.Type = 3;  
if (m_Anim_index == 35) ParryDesc.Type = 4;
```

```
// 패링 발생 위치와 기준 월드행렬 전달
```

```
ParryDesc.vPos = Pos;  
ParryDesc.WorldMatrix = m_pTransformCom->Get_WorldFloat4x4();
```

```
void CParry_Particle::Tick(_double TimeDelta)
```

```
{
```

```
지역변수 관리
```

```
    m_Life_Time += TimeDelta;
```

```
    // 빛이 강해졌다가 빠르게 줄어들도록 조절
```

```
    m_Light += TimeDelta * 5.f * m_Light_Dir;  
    if (m_Life_Time > 0.1f) m_Light_Dir = -1.f;
```

```
    // 사전 생성된 패링 전용 Point Light의 위치와 광량 갱신
```

```
    pGameInstance->Get_Light(PARRYPOINTLIGHT)->vLightPos = pos;  
    pGameInstance->Get_Light(PARRYPOINTLIGHT)->vLightDiffuse = _float4(m_Light, m_Light, m_Light, 1.f);
```

```
    // 파티클 크기도 시간에 따라 커졌다가 줄어들도록 조절
```

```
    m_Size_TimeAcc += TimeDelta;  
    m_Size += 4.0f * TimeDelta * m_Size_Dir;  
    if (m_Size_TimeAcc >= 0.2f) m_Size_Dir = -1.f;
```

```
    // LifeTime이 지나면 이펙트 제거
```

```
    if (m_Life_Time > PARRYLIFETIME) m_Is_Dead = true;
```

```
    // 실제 파티클 확산 계산은 공용 인스턴싱 버퍼 로직에 위임
```

```
    m_pVIBuffer_Point_InstanceCom->Tick(TimeDelta);
```

# [Thymesia Project] - Particle Instancing

## Feature

Parry, Spark, Dust 이펙트를

**Point Instancing** 기반 공용 버퍼 구조로 처리하는 **경량 파티클 시스템** 구현

## How it Works

이펙트 발생 시 개별 오브젝트 생성 대신 Point Instance Data 생성

위치, 방향, 크기, 수명 정보를 **공용 인스턴싱 버퍼**에 등록

Tick에서 각 인스턴스의 이동, 확산, 수명 감소를 갱신

동일한 점 기반 파티클을 **Instancing** 방식으로 일괄 렌더링

## Result

반복 생성되는 점 기반 파티클을 **경량 데이터 기반**으로 관리

Parry, Spark, Dust 이펙트를 하나의 공용 구조로 통합

**Draw Call**을 줄이고 렌더링 비용을 최소화



(Parry/Spark/Dust)\_Particle Event

InstanceDesc 설정

Point Instance Buffer 등록

Tick Update

Instanced Rendering

## InstanceDesc 기반 파티클 초기화

### Role

Parry, Spark, Dust 파티클의 초기 동작을

**InstanceDesc** 구조체 기반으로 정의하는 초기화 로직

### How it Works

InstanceDesc를 전달받아 파티클 초기 설정값으로 저장

각 인스턴스에 랜덤 속도, 이동 시간, 수명을 부여해 확산 편차 구성

**Instance Buffer**를 생성하고 GPU로 전달할 초기 데이터를 준비

Dir\_Type에 따라 Dust / Parry / Spark의 확산 방향을 다르게 계산

### Design Point

초기 파라미터와 방향 타입 조합으로

Parry, Spark, Dust 이펙트를 **공용** 구조에서 처리하도록 구현

```
typedef struct tagInstanceDesc
{ // 파티클 초기 설정 데이터를 정의하는 구조체
    _float3      vRange;
    _float       fMinSpeed, fMaxSpeed, SpeedAcc;
    _float       fMinMoveTime, fMaxMoveTime;
    _float       fMinLifeTime, fMaxLifeTime;
    _float4      vOffset, vSize;
    _float4x4     WorldMatrix;
    _uint        Dir_Type, Position_Type, Move_Type, Size_Type;
    _float       fGravity, fGravity_Time;
}INSTANCEDESC;                                     header
```

```
HRESULT CVertexBuffer Point_Instance::Initialize(void* pArg)
{
    // 전달받은 InstanceDesc를 기반으로 인스턴스 초기 상태 생성
    INSTANCEDESC InstanceDesc = *(INSTANCEDESC*)pArg;
    m_InstanceDesc = InstanceDesc;
    // 인스턴스별 상태 데이터 배열 생성
    m_pSpeed = new _float[m_iNumInstance];
    m_pMoveTime = new _float[m_iNumInstance];
    m_pLifeTime = new _float[m_iNumInstance];
    m_pDir = new _vector[m_iNumInstance];

    for (_uint i = 0; i < m_iNumInstance; ++i)
    { // 인스턴스별 데이터 랜덤 설정
        m_pSpeed[i] = rand() % _uint(InstanceDesc.fMaxSpeed - InstanceDesc.fMinSpeed + 1) + (_uint)InstanceDesc.fMinSpeed;
        m_pMoveTime[i] = rand() % _uint(InstanceDesc.fMaxMoveTime - InstanceDesc.fMinMoveTime + 1) + (_uint)InstanceDesc.fMinMoveTime;
        m_pLifeTime[i] = rand() % _uint(InstanceDesc.fMaxLifeTime - InstanceDesc.fMinLifeTime + 1) + (_uint)InstanceDesc.fMinLifeTime;
    }
}
```

```
INSTANCE_BUFFER_INIT
for (_uint i = 0; i < m_iNumInstance; ++i) {
    // 기준 위치를 중심으로 구 형태로 퍼지는 확산 방향 생성
    if (InstanceDesc.Dir_Type == DUST_SPHERE) {
        m_pDir[i] = { (_float)(rand() % 200 - 100), (_float)(rand() % 100 + 10), (_float)(rand() % 200 - 100)};
        m_pDir[i] += XMLoadFloat4(&InstanceDesc.vOffset);
        _vector Origin = XMLoadFloat4(&pInstanceVertices[i].vTranslation);
        m_pDir[i] = XMVector3Normalize(m_pDir[i] - Origin);
    } // 패링 방향 기준 좌측 확산 벡터 생성 및 월드 변환 적용
    else if (InstanceDesc.Dir_Type == PARRY_L) {
        m_pDir[i] = { -(_float)(rand() % 60 + 10), (_float)(rand() % 13 - 7), (_float)(rand() % 11 - 10)};
        m_pDir[i] += XMLoadFloat4(&InstanceDesc.vOffset);
        _vector Origin = XMLoadFloat4(&pInstanceVertices[i].vTranslation);
        m_pDir[i] = XMVector3Normalize(m_pDir[i] - Origin);
        m_pDir[i] = XMVector3TransformNormal(m_pDir[i], XMLoadFloat4x4(&InstanceDesc.WorldMatrix));
        XMStoreFloat4(&pInstanceVertices[i].vAxis, m_pDir[i]);
    }
}
```

# Instanced Particle Rendering

## Role

Tick에서 인스턴스 데이터를 갱신하고  
**Instancing** 방식으로 파티클을 렌더링하는 로직 구현

## How it Works

Tick에서 인스턴스별 위치 데이터 갱신

방향 벡터와 속도를 이용해 파티클 이동 처리

갱신된 데이터를 Instance Buffer에 반영

**DrawIndexedInstanced()**로 파티클 일괄 렌더링

## Design Point

Point Instance Buffer 기반으로 파티클 위치를 갱신하여  
다수의 점 파티클을 효율적으로 렌더링하도록 구현

```
void CVIBuffer_Point_Instance::Tick(_double TimeDelta)
{
    D3D11_MAPPED_SUBRESOURCE SubResource;
    // 갱신할 인스턴스 버퍼를 CPU에서 직접 수정할 수 있도록 매핑
    m_pContext->Map(m_pVBInstance, 0, D3D11_MAP_WRITE_NO_OVERWRITE, 0, &SubResource);

    for (size_t i = 0; i < m_iNumInstance; i++)
    {
        m_pMoveTimeAcc[i] += TimeDelta; // 인스턴스별 누적 이동 시간 갱신

        if (m_pMoveTimeAcc[i] > m_pMoveTime[i]) {
            // 일정 시간 이후 감속 계수를 줄여 파티클 속도를 점차 감소
            m_SpeedAcc -= TimeDelta * 10.f;
            if (m_SpeedAcc < 0.0f) m_SpeedAcc = 0.0f;
            // 최소 속도를 유지하며 감속 적용
            m_pSpeed[i] -= TimeDelta * m_SpeedAcc;
            if (m_pSpeed[i] < 2.f) m_pSpeed[i] = 2.f;
        }

        // 설정된 시간 이후에는 중력을 적용해 y축 방향 낙하 처리
        if (m_pMoveTimeAcc[i] > m_InstanceDesc.fGravity_Time)
            ((VTXINSTANCE*)SubResource.pData)[i].vTranslation.y -= m_InstanceDesc.fGravity * m_pSpeed[i] * TimeDelta;

        // 방향 벡터와 속도를 기반으로 인스턴스 위치 갱신
        ((VTXINSTANCE*)SubResource.pData)[i].vTranslation.x += XMVectorGetX(m_pDir[i]) * m_pSpeed[i] * TimeDelta;
        ((VTXINSTANCE*)SubResource.pData)[i].vTranslation.y += XMVectorGetY(m_pDir[i]) * m_pSpeed[i] * TimeDelta;
        ((VTXINSTANCE*)SubResource.pData)[i].vTranslation.z += XMVectorGetZ(m_pDir[i]) * m_pSpeed[i] * TimeDelta;
    }

    // 갱신된 인스턴스 데이터를 GPU가 사용할 수 있도록 매핑 해제
    m_pContext->Unmap(m_pVBInstance, 0);
}
```

Instance Data Update

```
HRESULT CVIBuffer_Instance::Render()
{
    m_pContext->DrawIndexedInstanced(m_iNumIndicesPerInstance, m_iNumInstance, 0, 0, 0);

    return S_OK;
}
```

N particle / 1 Draw Call



# [Thymesia Project] - Weapon Hit Detection (OBB Collision)

## Feature

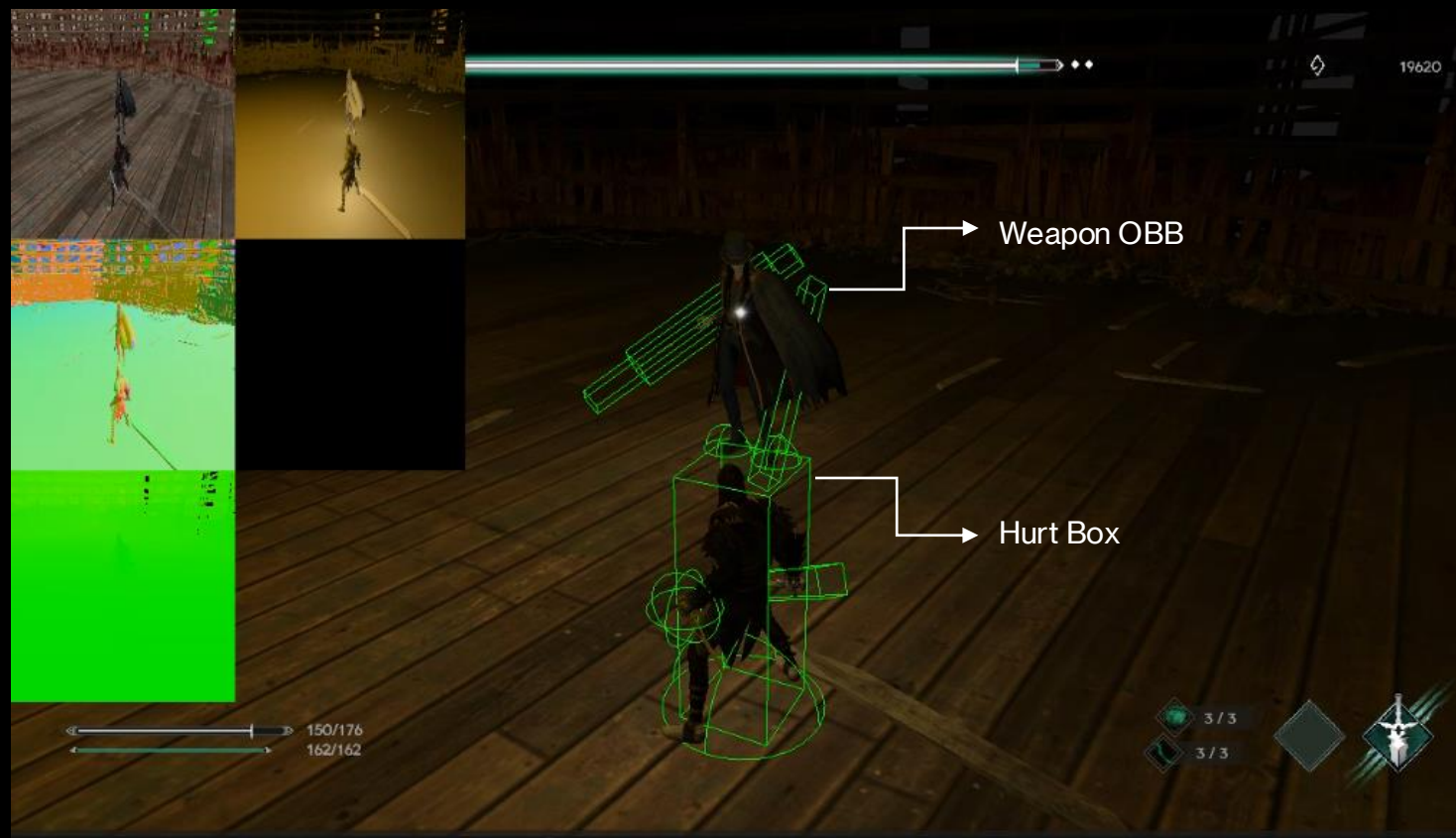
회전하는 무기 궤적을 반영하기 위해  
**OBB 기반 충돌 판정**으로 근접 공격 히트 검출 구현

## How it Works

무기의 Bone Transform을 기준으로 **Weapon OBB** 생성  
매 프레임 월드 행렬을 적용해 회전된 충돌 박스 갱신  
공격 애니메이션의 **특정 KeyFrame 구간**에서만 히트 판정 활성화  
Weapon OBB와 Hurt Box 간 충돌 여부를 검사해 공격 성공 처리

## Result

무기 방향과 자세 변화가 반영된 공격 판정 구현  
단순 축정렬 박스보다 자연스러운 근접 무기 충돌 처리  
공격 모션과 실제 히트 타이밍의 일관성 확보



# Weapon Hit Detection (OBB)

## Core Logic

**Weapon OBB 생성:** 무기 길이와 방향을 반영한 공격 판정 영역 구성

**Bone Transform 갱신:** 애니메이션에 따라 OBB 위치 · 회전 갱신

**KeyFrame Hit 판정:** 특정 공격 구간에서만 충돌 검사 수행

**Collision Test:** Intersect 함수로 Hurt Box 충돌 여부 판정

## Collision Algorithm

**Separating Axis Theorem (SAT)** 기반으로 구현

$|\text{dot}(C1 - C2, \text{axis})| > r1 + r2$

특정 축에서 위 조건이 성립하면 **분리축이 존재**하므로

충돌하지 않는 것으로 판단한다.

```
// Bone Transform을 기반으로 무기 월드행렬 계산
XMStoreFloat4x4(&m_WorldMatrix, m_pTransformCom->Get_WorldMatrix() *
    m_SwordDesc.pBone->Get_OffsetMatrix() *
    Remove_Scale(m_SwordDesc.pBone->Get_CombinedTransformMatrix()) *
    m_SwordDesc.pBone->Get_PivotMatrix() *
    m_SwordDesc.pParentTransform->Get_WorldMatrix());
// 갱신된 무기 행렬을 OBB Collider에 반영
m_pColliderCom->Tick(XMLoadFloat4x4(&m_WorldMatrix));
```

```
if (m_Anim_index == 18) {
    if (m_KeyFrame >= 98 && m_KeyFrame <= 112)
        Collision_ToPlayer();
}
```

→ 특정 KeyFrame 구간 검사

```
void CBoss1_Sword::Collision_ToPlayer()
{
    CGameInstance* pGameInstance = CGameInstance::GetInstance();
    Safe_AddRef(pGameInstance);

    _uint Level_Index = pGameInstance->Get_Level_Index();

    // 플레이어의 Hurt Box(AABB) Collider 가져오기
    CCollider* pTargetCollider = (CCollider*)pGameInstance->
        Get_Component(Level_Index, TEXT("Layer Player"), TEXT("Com_AABB"));
    // Weapon OBB와 Player Hurt Box 간 충돌 검사 수행
    m_pColliderCom->Intersect(pTargetCollider);
```

→ SAT 기반 충돌 검사

# [IRA Project]

## [Project]

IRA Recreation

## [Project Info]

Engine : DirectX9

Development Period : 30 Days

Team Size : 4

## [My Core Implementation]

Time Stop Skill System

Player Combat



# [IRA Project] - Time Stop Skill System

## Feature

Layer 단위 시간 제어를 통해 주변 오브젝트의 업데이트를 정지시키는 **Time Stop** Skill 구현

## How it Works

시간 정지 시 레이어가 관리하는 오브젝트에 0.f의 TimeDelta를 전달

적, 일반 오브젝트, 투사체 등의 Tick을 사실상 정지 상태로 처리

실제 TimeDelta는 레이어에 별도로 저장

플레이어와 관련된 **예외 객체**는 실제 TimeDelta를 참조해 계속 갱신

## Result

Time Stop 상태에서 주변 오브젝트의 움직임을 일괄 정지

플레이어와 스킬 관련 객체만 선택적으로 동작하도록 제어

전투 흐름을 일시적으로 제어하는 특수 스킬 구현

```
_int CLayer::Update_Layer(const _float & fTimeDelta)
{
    int iResult = 0;

    // 실제 TimeDelta 저장
    // Time Stop 예외 대상(플레이어/화살/스킬 이펙트 등)이 참조 가능하도록 유지
    m_fTimeDelta = fTimeDelta;

    for (auto iter = m_uMapObject.begin(); iter != m_uMapObject.end(); )
    {
        if (m_fTimeStop == false) {
            // 일반 업데이트
            iResult = iter->second->Update_GameObject(fTimeDelta);
        }
        else {
            // Time Stop 시 TimeDelta를 0으로 전달해 업데이트 정지
            iResult = iter->second->Update_GameObject(0.f);
        }
        ++iter;
    }
}
```

```
_int CPlayer::Update_GameObject(const _float& fTimeDelta)
{
    // Time Stop 예외 처리용 실제 TimeDelta 참조
    CLayer* pGameLogicLayer = Engine::Get_Layer(L"Layer_GameLogic");

    // 레이어에 저장된 실제 TimeDelta를 사용해 플레이어 업데이트
    _float Time = pGameLogicLayer->Get_TimeDelta();
}
```



# Thank you for reading

이종혁

jhorn3927@gmail.com

<https://youtu.be/MyqkYV1b1fc>

[github.com/Impersy](https://github.com/Impersy)

Code explanation available upon request